Oracle Database Backup & Recovery

Complete Guide to RMAN (Recovery Manager)

- Data Failures
- Recovery Objectives
- RMAN Introduction
- Architecture
- Backup Strategy
- Commands
- Incremental Backups
- Recovery Procedures

Data Failure Categories

Media Failure

- Faulty hardware
- Deleted data files
- Storage problems

User Error

- Accidental data deletion
- Wrong uploads
- Dropped tables

Network Failure

- Faulty interfaces

Application Errors

- Incorrect logic

- Misconfigurations
- Listener failures

- Wrong data changes

Disasters

- Floods, earthquakes
- Fires, water leaks
- Physical destruction

Key Recovery Objectives

Important Note

RTO and RPO are determined by the business, not DBAs, based on the value of the system. They differ depending on how critical the system is.

RTO (Recovery Time Objective)

Maximum downtime allowed

Example: 10 minutes

RPO (Recovery Point Objective)

Maximum data loss tolerated

Example: 1 day

Oracle Recovery Solutions Comparison

Solution	Purpose	License	Key Features
RMAN	Backups & recovery	Included	Partial backups, handles all failure types, slower recovery
RAC	High availability	Separate, expensive	Active clustering, zero downtime, hardware fault tolerance
Data Guard	Disaster recovery	Separate (Active DG optional)	Redo log sync, standby database, read-only access with license
Flashback DB	Point-in-time recovery	Enterprise Manager	Rewinds database using flashback logs, partial recovery

Recovery Plan Phases

1⃣ Define

Establish RTO and RPO targets

2⃣ Implement

Acquire licenses and hardware, configure solutions

3⃣ Monitor

Observe system performance and detect issues

4⃣ Adjust

Optimize procedures based on testing and feedback

Best Practices

- Prepare for all types of failures, regardless of cost
- Align recovery solutions with system value and budget
- Regularly test recovery processes
- Keep detailed documentation of both backup and recovery steps

What is RMAN?

Recovery Manager (RMAN): Oracle's built-in utility for database backup and recovery.

✓ **Included with Oracle Standard and Enterprise Editions**
No additional license required

Runs outside the database
But integrates seamlessly with it

Command-line interface
With scripting capabilities

Key Benefits

⚡ Efficient Backups

Skips unused data blocks to save time and space

Incremental Backups

Only backs up changes since the last backup

Platform Independent

Works across different operating systems

Encryption

Secures backup files

Database Cloning

Create copies offline or online

Tape Compatibility

Works with Oracle Secure Backup, NetBackup, TSM, etc.

☁ Cloud Support

Direct backups to Oracle Cloud or Amazon S3

Compression

Reduces backup file sizes

Corruption Detection

Detects corrupted blocks during backup

RMAN Architecture Components

Target Database

The database being backed up or recovered

Recovery Catalog Database

Stores backup metadata Optional

Backup Storage

Disks, cloud storage, tape libraries, or ZDLRA

Duplicate Database

Optional copy created from the target

Command Types

- **Interactive commands:** Entered directly at the RMAN prompt
- **Script commands:** Run from OS scripts or scheduled jobs
- **EM/Third-party integration:** Via Oracle Enterprise Manager or backup software

Physical vs Logical Backups

Type	Tool	Description	Recommendation
Physical	**RMAN**	Copies actual data blocks	Recommended
Logical	**Data Pump**	Exports data as SQL commands	Not for backups

RMAN vs User-Managed Backups

✓ RMAN Preferred

- Fully integrates with Oracle
- Supports online/incremental backups
- Compatible with Enterprise Manager

⚠ User-Managed

- Uses SQL or OS commands only
- Lacks RMAN features
- Limited functionality

Recommended

Not recommended

Getting Started Commands

Start RMAN:

```
rman
```

Connect to target database:

```
CONNECT TARGET username/password@database
```

Configure settings:

```
CONFIGURE ...;
```

Issue commands:

```
BACKUP DATABASE;
RESTORE DATABASE;
RECOVER DATABASE;
```

RMAN Command Language Overview

Key Concept

RMAN has its own command language, separate from the operating system.

Command Types

Stand-alone Commands

Executed individually at the RMAN prompt

```
BACKUP DATABASE;
```

Type the command with a semicolon (;) and press Enter. RMAN executes it immediately.

Job Commands (RUN Blocks)

Grouped together and executed sequentially

```
RUN {
    command1;
    command2;
    command3;
}
```

If any command fails, RMAN stops executing remaining commands. Effects of executed commands are not undone.

Examples of Stand-Alone Commands

- RMAN commands start with a keyword, followed by one or more arguments
- Commands must end with a semicolon to execute
- Commands can span multiple lines
- Comments can be added using the hash symbol (#) - everything after # is ignored

Example command structure:

```
BACKUP DATABASE; -- BACKUP is the command, DATABASE is the argument -- Multi-line command: BACKUP DATABASE ;
```

RMAN Persistent Settings

RMAN behavior is controlled by configuration settings stored in the control file and optionally in the recovery catalog.

Settings persist across RMAN sessions and define parameters like:

- Channel allocation
- Parallelism
- Default device type
- Backup retention policy

```
View all settings:
SHOW ALL;
```

```
View specific setting:
SHOW <setting>;
```

```
Query via SQL:
SELECT * FROM V$RMAN_CONFIGURATION;
```

Changing RMAN Settings

Use the CONFIGURE command to change settings. Settings take effect for all subsequent operations unless overridden in a RUN block.

```
Configure parallelism:
CONFIGURE PARALLELISM 3 FOR DEVICE TYPE sbt_tape;
```

```
Clear a configuration:
CONFIGURE BACKUP OPTIMIZATION CLEAR;
```

Modes of Issuing RMAN Commands

Mode	Description	Best For
Interactive	Type commands directly at the prompt	One-time operations, reports, specific backups/recoveries
Batch	Commands written in a file, run using `rman @script_file`	Automated jobs
Pipe Interface	Commands via DBMS_PIPE PL/SQL package	External applications (rarely used by DBAs)

Backup Strategy

Backup Options

Whole Database Backup

- Includes all data files
- Includes control file
- Backs up entire database

Partial Database Backup

- Specific tablespaces only
- Specific data files only
- May include/exclude control file

Backup Types

Type	Description	Requirements
Full Backup	Backs up all used data blocks in specified files	-
Incremental Backup	Backs up only blocks changed since previous backup	Requires Level 0 backup first

Important Note

Full backup ≠ Whole backup

- **Full backup:**

  backs up all used blocks

- **Whole backup:**

  backs up the entire database

- Both can be combined (e.g., full backup of the entire database)

Backup Modes

❄ Offline (Cold) Backup

Status: Database closed

Consistency: SCN numbers in all data file headers match the control file

Consistent

Online (Hot) Backup

Status: Database open

Consistency: Data files may not be synchronized with control files

Inconsistent

RMAN Backup File Types

Feature	Backup Set	Image Copy
Contains	Used blocks only	Full copy of data files
Storage	Disk or tape	Disk only
Restore granularity	Must extract from backup set	Can restore individual file directly
Space efficiency	Smaller	Larger
Use for standby DB	Not directly	Yes

Recommendation

- Use **backup sets** for regular backups
- Use **image copies** if granular restore or standby DB setup is needed

RMAN Backupable Files

✓ Can Backup

- Data files
- Control files
- Server parameter files
- Archived redo logs

✗ Cannot Backup

- Online redo logs
- Network configuration files
- Password files
- Keystore files

- RMAN backups

Must backup manually

RMAN BACKUP Command

Backup Types by Consistency

Inconsistent Backup (Hot)

- Taken while database is open
- SCN numbers may not match control file
- Can only be done in ARCHIVELOG mode
- Recovery requires applying archived redo logs

❄ Consistent Backup (Cold)

- Taken while database is mounted after clean shutdown
- SCN and checkpoint timestamps synchronized
- Can be done in ARCHIVELOG or NOARCHIVELOG mode
- In NOARCHIVELOG mode, changes after backup are lost if failure occurs

Performing Backups

```
Online Backup (Open Database):
BACKUP DATABASE PLUS ARCHIVELOG;
```

Backs up data files and archived redo logs. RMAN takes two backups of archived redo logs (before and after data file backup) to ensure recoverability.

```
Offline Backup (Mounted Database):
```

```
BACKUP DATABASE;
```

Requires shutdown and mount before backup. Database can be started after backup completes.

Partial Backups

```
Backup specific tablespaces:
BACKUP TABLESPACE users, tools;
```

```
Backup data files by number:
BACKUP DATAFILE 1,2,3,4;
```

```
Backup data files by full path:
BACKUP DATAFILE '/u01/oradata/orcl/users01.dbf';
```

Backup File Locations and Naming

- **Default storage:** FRA (Flash Recovery Area)
- **FORMAT option:** Specify path and file name pattern

```
Custom format:
BACKUP DATABASE FORMAT '/u02/orcl_%U';
```

`%U` generates a unique string to ensure unique file names

```
ASM diskgroups:
BACKUP DATABASE FORMAT '+DGNAME';
```

ASM automatically generates unique names

Backing Up Control File and SPFILE

Control File Backup Methods

1 Automatic Backup (Autobackup)

```
CONFIGURE CONTROLFILE AUTOBACKUP ON;
```

ON by default from 12.2 onward

Occurs after every RMAN backup automatically

2 Manual Backups

```
BACKUP DATABASE INCLUDE CURRENT CONTROLFILE;
```

```
BACKUP CURRENT CONTROLFILE;
```

```
BACKUP AS COPY CURRENT CONTROLFILE;
```

3 Indirect Method

Backing up data file #1 automatically includes the control file and SPFILE

Snapshot Control File

RMAN creates a Snapshot Controlfile when backing up the control file or syncing with the recovery catalog.

Key Points:

- Used internally by RMAN for consistency

- Not the same as the Autobackup control file

```
Show snapshot control file location:
SHOW SNAPSHOT CONTROLFILE NAME;
```

```
Configure snapshot control file location:
CONFIGURE SNAPSHOT CONTROLFILE NAME TO '/path/controlfile_snap.f';
```

Trace File Backup

Generates SQL statements to recreate control file if all backups are lost.

```
Create trace backup:
ALTER DATABASE BACKUP CONTROLFILE TO TRACE;
```

Important Notes:

- Saved under $DIAGNOSTIC_DEST directory
- Cannot be executed directly; must follow the instructions in the trace file
- Rarely needed if Autobackup and routine RMAN backups are in place

SPFILE Backup

- SPFILE contains initialization parameters for the database
- Autobackup of control file includes a backup of SPFILE automatically

```
Manual backup command:
BACKUP SPFILE;
```

Trace File Backup

Generates SQL statements to recreate control file if all backups are lost

```
ALTER DATABASE BACKUP CONTROLFILE TO TRACE;
```

Saved under `$DIAGNOSTIC_DEST` directory

> **Note:**
>
> Cannot be executed directly; must follow the instructions in the trace file.

Archivelog Backup Commands

BACKUP ARCHIVELOG Command

```
All available archivelogs:
BACKUP ARCHIVELOG ALL;
```

```
Delete after backup:
BACKUP ARCHIVELOG ALL DELETE INPUT;
```

```
Backup by SCN range:
BACKUP ARCHIVELOG FROM SCN 1000 UNTIL SCN 2000;
```

```
Backup by sequence number:
BACKUP ARCHIVELOG SEQUENCE 100 UNTIL 200;
```

```
Backup by time:
```

```
BACKUP ARCHIVELOG FROM TIME "SYSDATE-1";
```

BACKUP DATABASE PLUS ARCHIVELOG

Simplifies backup process by including archived redo logs automatically

Internal Steps RMAN Performs:

1. Runs `ALTER SYSTEM ARCHIVE LOG CURRENT`
2. Runs `BACKUP ARCHIVELOG ALL`
3. Backs up the specified objects
4. Runs `ALTER SYSTEM ARCHIVE LOG CURRENT` again
5. Backs up any remaining archived redo logs

```
BACKUP DATABASE PLUS ARCHIVELOG;
```

Recommendation:

Use `BACKUP DATABASE PLUS ARCHIVELOG` for online backups to ensure recoverability.

RMAN Backup as Image Copies

Key Concept:

Use AS COPY in the BACKUP statement to create an image copy.

Supported Objects

- Data files

- Archived redo logs
- Control files

Behavior

✅ Benefits

- Bit-by-bit copy of the target file
- RMAN validates blocks while copying (better than OS-level copy)
- Registers output files in RMAN repository automatically
- Assigns unique names unless specified

Specifying Location

```
Method 1: DB_FILE_NAME_CONVERT
BACKUP AS COPY DATAFILE '/u01/oradata/users/users01.dbf' TO
'/backups/users_ts/users01.dbf';
```

```
Method 2: FORMAT clause
BACKUP AS COPY DATAFILE 'users_tbs01.dbf' FORMAT '/BACKUP/users01.dbf';
```

```
Optional: NOCHECKSUM (speeds up backup, but skips block corruption check)
BACKUP AS COPY DATAFILE 1 NOCHECKSUM;
```

Incremental Backups

Overview

Only backs up changed blocks since the last backup, not the entire database.

⚡ Benefits

- Faster than full backups
- Less storage space required
- Lower I/O bandwidth needed
- Can back up NOLOGGING objects

Level 0 and Level 1 Backups

Level	Description	File Types
Level 0	Full backup of all database blocks. Forms the base for all Level 1 backups.	Backupsets or image copies
Level 1	Backs up only changed blocks	Backupsets only (not image copies)

Differential vs Cumulative

Feature	Differential	Cumulative
What it backs up	Changes since last incremental backup	Changes since last Level 0 backup

Feature	Differential	Cumulative
Backup size	Smaller	Larger
Recovery	Reads all Level 1 backups + Level 0	Reads only most recent Level 1 + Level 0
Advantage	Saves storage space	Faster recovery

Recommendation:

Differential backups are preferred in large databases to save disk space.

Incremental Backup Commands

Default (Differential):

```
BACKUP INCREMENTAL LEVEL 1 DATABASE;
```

Cumulative:

```
BACKUP INCREMENTAL LEVEL 1 CUMULATIVE DATABASE;
```

Incrementally Updated Backups

RMAN allows updating Level 0 backups with Level 1 backups. This reduces storage usage and allows maintaining an always up-to-date backup.

Benefits:

- Reduces storage requirements
- Maintains an always current backup

- Faster recovery time

Retention Policy Note:

Keep retention policy at redundancy 1 when using incrementally updated backups. Using window-based or higher redundancy can cause FRA to fill because all incremental backups and archived logs must be kept to satisfy policy.

Block Change Tracking (BCT)

Speeds up incremental backups by tracking changed blocks in a tracking file.

Enable BCT:

```
ALTER DATABASE ENABLE BLOCK CHANGE TRACKING USING FILE
'/u01/oradata/bct_file.bct';
```

Disable BCT:

```
ALTER DATABASE DISABLE BLOCK CHANGE TRACKING;
```

Move BCT file:

```
ALTER DATABASE RENAME BLOCK CHANGE TRACKING FILE TO '<new_location>';
```

BCT Details

How it works:

- Oracle maintains a binary BCT file containing addresses of changed blocks
- RMAN uses this file instead of scanning all blocks during an incremental backup
- Can track up to 8 incremental backups
- After 7 incremental backups, take a full Level 0 backup
- BCT file grows in 10 MB increments; even for heavy workloads, it remains small

- Not suitable if >20% of database changes frequently
- Not enabled by default
- RMAN does not backup or restore BCT file
- In RAC, BCT file should be on shared storage

Monitoring BCT

```
Query BCT information:
SELECT FILE#, STATUS, BLOCKS, BYTES, FILE_NAME FROM V$BLOCK_CHANGE_TRACKING;
```

Effectiveness check: Ratio of blocks read by RMAN vs total blocks (lower ratio = more effective)

Best Practices for Incremental Backups

Weekends

Take Level 0 incremental backup

Weekdays

Take Level 1 incremental backups (off-peak hours)

Backup Type

Prefer differential incremental backups over cumulative

⚡ Performance

Enable Block Change Tracking for large databases

Backup Retention Policy

Retention policy determines which backups RMAN keeps or marks as obsolete.

Types of Retention Policies

Redundancy-Based

Keeps a fixed number of backups

```
CONFIGURE RETENTION POLICY TO
REDUNDANCY 3;
```

Keeps last 3 backups, older ones marked obsolete

Window-Based (Time-Based)

Keeps backups for a specified period

```
CONFIGURE RETENTION POLICY TO
RECOVERY WINDOW OF 7 DAYS;
```

Allows recovery to any point in last 7 days

Important Notes:

- Only one retention policy can be active at a time
- Default redundancy is 1 backup
- Always configure a retention policy to avoid storage issues in FRA

Disabling Retention Policy

```
CONFIGURE RETENTION POLICY TO NONE;
```

Not Recommended:

RMAN won't consider any backup obsolete. Especially problematic when using FRA.

RMAN Recovery Overview

Types of Database Failures

Physical Failures

Problems with actual database files (lost or corrupted)

Logical Failures

Issues caused by applications or users (accidental or intentional data changes)

Reasons for Database File Loss

Human Error

Accidental deletion or overwriting by DBA

Application Error

Faulty scripts corrupt or delete files

Intentional Harm

Malicious acts like ransomware

Media Failure

Disk or controller issues

Hardware Failure

Server goes offline, files become inaccessible

RMAN Recovery Concepts

Command	Purpose	Description
RESTORE	Creates database file from backup	Restores file to backup point, not the failure time
RECOVER	Applies changes	Uses archived redo logs, incremental backups, and online redo logs

Recovery Types:

- **Complete Recovery:**

  Up to failure point

- **Incomplete / Point-in-Time Recovery (PITR):**

  Up to a specified time before failure

RESTORE Command Key Points

RESTORE Details

- Can be used when DB is started, mounted, or open (file to restore must be offline if DB is open)
- Can restore datafiles, control files, SPFILE, archived redo logs (not temporary tablespaces)
- Restores to original location by default; can specify a different location
- Chooses the most recent backup by default; can select specific backup sets or copies
- Can exclude tablespaces, check corruption without restoring, or list backup files

RECOVER Command Key Points

RECOVER Details

- Applies changes to restored data files
- Cannot be used if database is in NOARCHIVELOG mode
- Supports complete recovery and incomplete/PITR
- Can apply incremental backups to image copies
- Can recover corrupted blocks

Preparing for Recovery

Preparation Steps (need-based, not always required):

- Identify files that need recovery
- Determine DBID if restoring control file or SPFILE
- Validate backups for availability and corruption
- Restore archived redo logs if needed (usually automatic)
- Provide password for encrypted backups

Critical vs Non-Critical Files

✕ Non-Critical Files

Can be lost without crashing the DB

- Multiplexed redo logs
- Index tablespaces

Critical Files

Loss causes database crash

- Control files
- System tablespace datafiles

- Init parameter files
- Inactive temp files

When Recovery is Needed

Identifying Missing Files

Oracle errors like ORA-00205 indicate:

- Missing control file
- Missing data file

Validate Missing Files

Use VALIDATE command on the database, PDB, tablespace, or specific data file.

```
Validate database:
VALIDATE DATABASE;
```

```
Validate specific datafile:
VALIDATE DATAFILE 5;
```

Checking Backup Availability

```
RESTORE PREVIEW (shows backup files RMAN will access):
RESTORE DATABASE PREVIEW;
```

Does not check physical presence of files

```
RESTORE VALIDATE HEADER (validates headers and checks location):
RESTORE DATABASE VALIDATE HEADER;
```

Ensures backup files exist before starting a restore

Validating Backup Integrity

Validation process checks for corrupted blocks in backups.

```
Method 1: RESTORE ... VALIDATE
RESTORE DATABASE VALIDATE;
```

```
Method 2: VALIDATE command
VALIDATE BACKUPSET ALL;
```

> **Note:**
>
> Large backups may take time; can be skipped in urgent scenarios.

Whole Database Recovery

NOARCHIVELOG Mode

> Cannot recover to the point of failure unless incremental backups exist.

```
Recovery Steps:
-- Start database in MOUNT state STARTUP MOUNT; -- Restore database RESTORE
DATABASE; -- Recover database (typically does nothing in NOARCHIVELOG mode)
```

```
RECOVER DATABASE; -- Open with RESETLOGS ALTER DATABASE OPEN RESETLOGS;
```

ARCHIVELOG Mode

Can perform complete or incomplete recovery. Most production databases use ARCHIVELOG mode.

```
Recovery Steps:
STARTUP MOUNT; RESTORE DATABASE; RECOVER DATABASE; ALTER DATABASE OPEN;
```

Recovery to a New Location

```
RUN { SET NEWNAME FOR DATAFILE 1 TO '/new/location/system01.dbf'; SET NEWNAME
FOR DATAFILE 2 TO '/new/location/sysaux01.dbf'; RESTORE DATABASE; SWITCH
DATAFILE ALL; RECOVER DATABASE; }
```

User Tablespace Recovery

Only affects the specific tablespace, not SYSTEM, SYSAUX, UNDO, or TEMP tablespaces.

```
Recovery Steps:
-- Take tablespace OFFLINE ALTER TABLESPACE users OFFLINE; -- Restore
tablespace RESTORE TABLESPACE users; -- Recover tablespace RECOVER TABLESPACE
users; -- Bring tablespace ONLINE ALTER TABLESPACE users ONLINE;
```

Recovery by Switching Data Files

Instead of restoring datafiles from backup sets, switch the database to use existing data file image copies.

> **Advantage:**
>
> Faster than restoring from backups because RMAN updates the control file instead of creating new files.

Scenario: All Database Datafiles Lost

```
-- Mount the database STARTUP MOUNT; -- Switch to data file copies SWITCH DATABASE TO COPY; -- Recover database RECOVER DATABASE; -- Open database ALTER DATABASE OPEN;
```

Scenario: Single User Tablespace Datafile Lost

```
-- Take datafile offline ALTER DATABASE DATAFILE 5 OFFLINE; -- Switch to image copy SWITCH DATAFILE 5 TO COPY; -- Recover datafile RECOVER DATAFILE 5; -- Bring datafile online ALTER DATABASE DATAFILE 5 ONLINE;
```

Point-in-Time Recovery (PITR)

Restore the database or parts of it to a previous state (e.g., accidental data changes or missing archive logs).

Levels of PITR

Database Level

DBPITR - Full database recovery

Tablespace Level

TSPITR - Single tablespace recovery

Table Level

Recover specific tables

Recovery Point Options

- SCN (System Change Number)
- Redo log sequence number
- Restore point
- Timestamp

Oracle converts all to SCN internally

DBPITR Requirements

- Database must be in ARCHIVELOG mode
- Backups of all datafiles must exist before the required recovery SCN

DBPITR Procedure

Step-by-Step Recovery:

```
-- Shutdown database SHUTDOWN IMMEDIATE; -- Start RMAN and run recovery RUN {
SET UNTIL TIME 'yyyy-mm-dd hh24:mi:ss'; RESTORE DATABASE; RECOVER DATABASE; }
-- Open with RESETLOGS ALTER DATABASE OPEN RESETLOGS;
```

RESETLOGS Required:

After DBPITR, the database must be opened with RESETLOGS, which:

- Archives online redo logs if present

- Clears or recreates online redo logs
- Resets log sequence to 1
- Updates all files with a new RESETLOGS SCN
- Creates a new database incarnation

Incarnation Concept

Each RESETLOGS operation creates a new database incarnation.

Importance of Incarnations:

- Archived redo logs and database must have matching incarnations for recovery
- Incarnation affects which backups can be restored
- Flashback database can rewind to a previous incarnation if needed

DBID (Database Identifier)

Required if restoring SPFILE or control file.

Methods to Obtain DBID

- Check past RMAN log files
- Look at AUTOBACKUP control file name (DBID is in the filename)

Recommendation:

Save DBID externally with backup files.

Optional: Open DB Read-Only Before RESETLOGS

Useful if the exact recovery point is unclear.

Modified Procedure:

```
-- 1. Shutdown database SHUTDOWN IMMEDIATE; -- 2. Start NOMOUNT and set DBID STARTUP NOMOUNT; SET DBID = <dbid_value>; -- 3. Restore control file from AUTOBACKUP RESTORE CONTROLFILE FROM AUTOBACKUP; -- 4. Mount database ALTER DATABASE MOUNT; -- 5. Restore and recover to desired point RUN { SET UNTIL TIME 'yyyy-mm-dd hh24:mi:ss'; RESTORE DATABASE; RECOVER DATABASE; } -- 6. Open READ ONLY to verify data ALTER DATABASE OPEN READ ONLY; -- 7. Verify data, repeat steps if necessary -- 8. Open with RESETLOGS once satisfied SHUTDOWN IMMEDIATE; STARTUP MOUNT; ALTER DATABASE OPEN RESETLOGS;
```

Warning:

This method overwrites the current control file. Use with caution.

Advanced RMAN Features

Using TAGs

Assign a meaningful name to a backup set or image copy for easier management and recovery.

```
BACKUP DATABASE TAG 'MONTH_WHOLE_BACKUP';
```

Best Practice:

Always use TAGs in production environments for better backup management.

Backup Storage Devices

Device Type	Description	Requirements
Disk	RMAN saves backups to disk storage	Default
Tape (SBT)	System Backup to Tape	Requires third-party software

Set default device type:

```
CONFIGURE DEFAULT DEVICE TYPE TO DISK;
```

Override at command level:

```
BACKUP DATABASE DEVICE TYPE SBT;
```